

Alyssia MONTTOY

4TS1

In422: Real Time Embedded Systems

Final Assignment – Report

Jasdeep SINGH



Deadline: April 28th, 2026

Presentation of the subject

During this project, we will see if a set of tasks is schedulable or not.

Given the following task set:

Task	C (c'est quoi ?)	T_i (deadline)
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

C_i is the execution time of the task τ_i , while T_i is the period of repetition of the task.

Determination of the execution time of C_1

Code for Task_1.c :

```
1  #include <stdlib.h> // pour le rand()
2  #include <stdio.h>
3
4  int main(void) {
5      int a = rand() % 900000 + 100000;
6      int b = rand() % 900000 + 100000;
7      int product = a * b;
8      printf("Product: %d\n", product);
9
10     return 0;
11 }
```

It is a C code that generates two large random numbers and multiplies them.

(Code extract 1: Task_1.c)

The goal will be to determine the execution time of the code. For that, we create a python file.

The program WCET.py

1. Open and compile the file Task_1.c
2. Stores the information of the starting hour a variable start_time
3. Executes the code in the file
4. Stores the information of the ending hour a variable end_time
5. Returns the execution time by calculating the difference (end_time - start_time)

The WCET function is called several times, which can be changed by varying nb_iter. Here we will start by iterating 1000 times.

```

Min : 0.0010543500029598363 s
Max : 0.0038664579988108017 s
Q1 : 0.001266058003238868 s
Q2 : 0.0013565170011133887 s
Q3 : 0.001450987001589965 s
Mean : 0.0013794938167600776 s
WCET is 3 ms

```

We get the values. We notice that 1,000 iterations is not enough because, between two runs of the program, the average values vary. So we go to 10,000 iterations and the results are much more stable.

The curve representing the distribution of these run-time values is a Gaussian.

D'après les résultats ci-dessus on va définir $C_1 = 3 \text{ ms}$, the execution time of our task τ_1

Schedule

A system is said to be schedulable if it allows all the requested tasks to be carried out without missing a deadline. Theoretically, we can determine schedulability:

- **Step 1: Calcul of U_i the CPU Utilization for task τ_i :** $U_i = \frac{C_i}{T_i}$

	τ_1	τ_2	τ_3	τ_4	τ_5	τ_6	τ_7
C_i/T_i	4/10	2/10	2/20	2/20	2/40	2/40	3/80
U_i	0.4	0.2	0.1	0.1	0.05	0.05	0.0375

We calculate the percentage of use of a task by the CPU. Task τ_i will request the use of the CPU for a duration C_i at a frequency of $1/T_i$ Hz. He is therefore using the CPU up to C_i/T_i

- **Step 2 : Calcul of the Total Utilization**

Total Utilization = 0.4 + 0.2 + 0.1 + 0.1 + 0.05 + 0.05 + 0.375 = 0.9375 < 1

To be schedulable it is necessary (but not sufficient) to have a Total Utilization less than 1. Here, the necessary condition is met; we will review it in detail if the system is schedulable.

Scheduling algorithm

Scheduling is the order in which our system is told to execute. There are different types of schedules. For real-time systems, the schedules we find are:

Earliest Deadline First (EDF): The task with the closest deadline runs first. This minimizes the risk of missed deadlines. In this case, the necessary condition previously calculated is also sufficient.

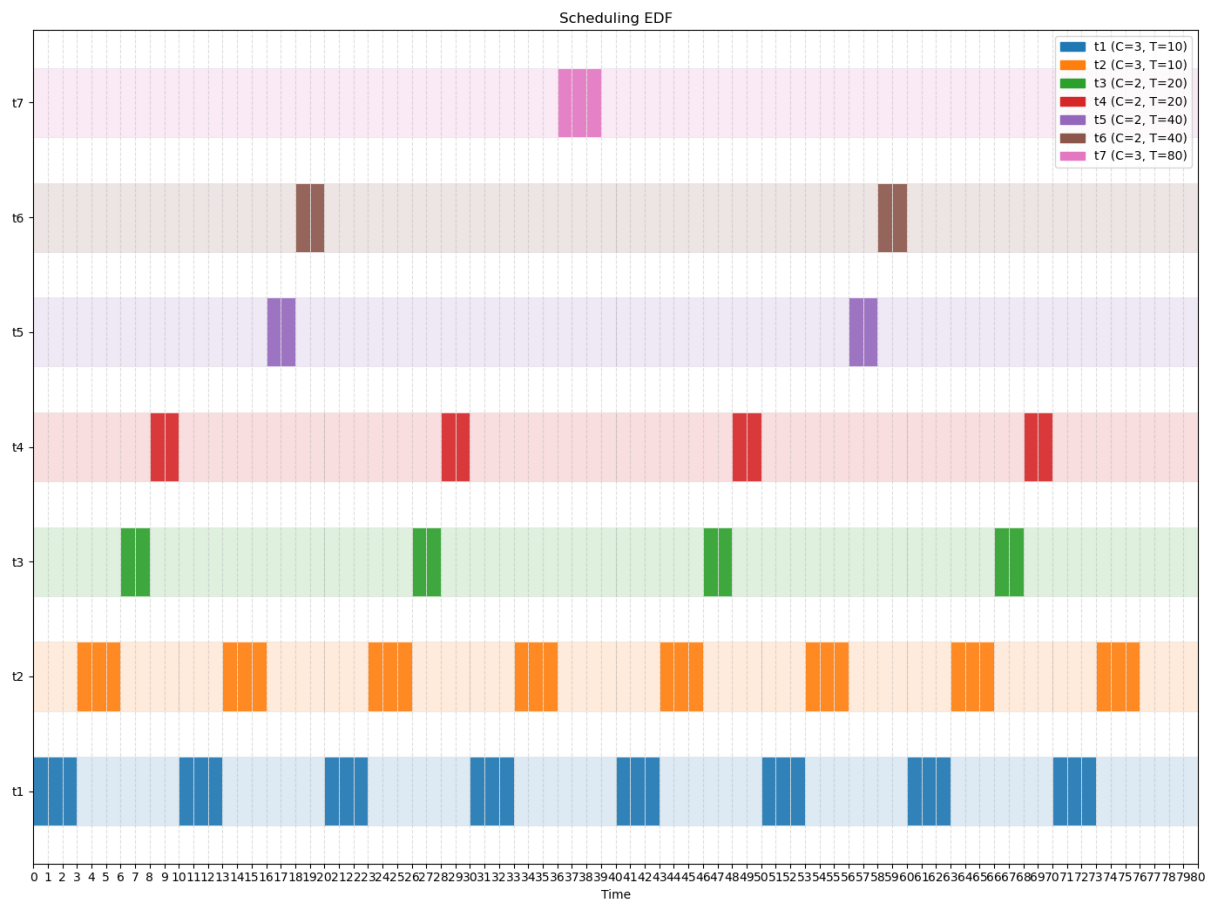
Rate Monotonic Scheduling (RMS) : we run the tasks one after another in order. Still in this order, if task 7 has finished and task 1 is not ready, then we wait for it even though another one is ready. It doesn't optimize the use of the processor, but it's the simplest algorithm.

To ensure that this algorithm is schedulable, here we calculate the hyperperiod. This is the smallest time interval after which a task set will repeat in the same way.

Hyperperiod = $LCM(T_1, T_2, T_3, T_4, T_5, T_6, T_7)$, where LCM is the Least Common Multiple.

Hyperperiod = $LCM(10, 20, 40, 80) = 80$

So, we'll check for 80 ms if everything goes well. With a Python program, we can do it:



And we see that up to 80ms, tasks follow one another without missed deadlines, so the system is well schedulable.

Minimize the total waiting time

In this part we will search which scheduling minimizes the time execution of our task set. We implement our task system, as well as the two algorithms and compare which one gives a minimum waiting time.

In the Python program, we add a line to indicate the waiting time. The time when a task is ready but unused.

In the case of our program, it's quite simple: the periods are multiples of one another, so the waiting time is the same:

```
Total waiting time for RMS : 184.0  
Total waiting time for EDF : 184.0
```

But in practice, EDF allows for better optimization of CPU use, and leads to fewer missed deadlines.

Difficulties encountered

We had a quick introduction to Linux but didn't know much about it. So having to put files in the Linux part is not a reflex at all and I took a lot of time to understand why my code didn't want to work. The base libraries on Pythons were no longer present when we went through the WSL.

Similarly, we discovered the C last year and only used it during 1 project. Although in this Final Assignment we only did basic codes in C, it was a bit disturbing.

Having to work with these tools was a bit of a challenge. But once you got into it, things went very quickly.